Initialization Strategies for Deep ReLU Networks: Geometry Preservation vs. Gradient Stability

Itay Ofer

March 2026

Contents

Chapter 1

Introduction and Mathematical Framework

1.1 Problem Statement

Consider a deep feed-forward ReLU network with L hidden layers. Layer l maps $\mathbb{R}^{d_{l-1}} \to \mathbb{R}^{d_l}$ via

$$\boldsymbol{h}^{(0)} = \boldsymbol{x}, \qquad \boldsymbol{h}^{(l)} = \mathrm{ReLU}\big(W^{(l)}\,\boldsymbol{h}^{(l-1)}\big), \quad l = 1, \dots, L, \tag{1.1}$$

where $W^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ and $\mathrm{ReLU}(z) = \max(0, z)$ is applied element-wise. In general, the weight matrices need not be square. For simplicity, in this work we study constant-width networks ($d_l = d$ for all l), but the analysis generalizes to non-square layers.

We study two desiderata for the weight matrices $\{W^{(l)}\}$:

(i) **Geometry preservation.** Class separability of the input data should survive the composition of L layers. Formally, for data pairs $(\boldsymbol{x}_i, \boldsymbol{x}_j)$, we would like

$$\mathbb{E}\Big[\big\langle \mathrm{ReLU}(W\boldsymbol{x}_i),\ \mathrm{ReLU}(W\boldsymbol{x}_j)\big\rangle\Big] \ \approx\ \langle \boldsymbol{x}_i,\ \boldsymbol{x}_j\rangle.$$

(ii) **Gradient stability.** The gradient magnitude $\|\partial \mathcal{C}/\partial W^{(l)}\|$ should be approximately the same for every layer l, so that all layers receive a useful learning signal.

The central finding of this work is that these two objectives are in *structural conflict* when the nonlinearity is ReLU.

1.2 The Arc-Cosine Kernel (Summary)

The arc-cosine kernel framework [1] characterizes the expected inner product of post-ReLU outputs. These results were discussed previously with the advisor; we state them here without proof for reference.

Let $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ have i.i.d. rows $\boldsymbol{w}_k \sim \mathcal{N}(\mathbf{0},\, \sigma^2 I)$. For two unit-norm inputs with angle $\alpha = \arccos\big(\langle \boldsymbol{x}_i, \boldsymbol{x}_j\rangle\big) \in [0, \pi]$:

Theorem 1.1 (Arc-cosine kernel, first order)**.**

$$K(\alpha) \ =\ \frac{1}{d_{\text{out}}}\,\mathbb{E}\Big[\big\langle \mathrm{ReLU}(W\boldsymbol{x}_i),\ \mathrm{ReLU}(W\boldsymbol{x}_j)\big\rangle\Big] \ =\ \frac{\sigma^2}{2\pi}\big(\sin\alpha + (\pi - \alpha)\cos\alpha\big). \tag{1.2}$$

The normalized kernel $\hat{K}(\alpha) = K(\alpha)/K(0) = \frac{1}{\pi}(\sin\alpha + (\pi - \alpha)\cos\alpha)$ gives the expected cosine similarity of the outputs. The output angle after one layer is:

$$\alpha' = \arccos\big(\hat{K}(\alpha)\big) = \arccos\left(\frac{\sin\alpha + (\pi - \alpha)\cos\alpha}{\pi}\right). \tag{1.3}$$

Since $\hat{K}(\alpha) > \cos\alpha$ for all $\alpha \in (0, \pi)$, the output angle is always smaller than the input angle:

Proposition 1.2 (Systematic angle contraction)**.**

$$D(\alpha) = \hat{K}(\alpha) - \cos\alpha = \frac{1}{\pi}\big(\sin\alpha - \alpha\cos\alpha\big) > 0 \quad \textit{for all } \alpha \in (0, \pi). \tag{1.4}$$

This means **every pair of distinct vectors becomes more similar after one ReLU layer**. Over L layers this compounds: angles contract toward 0, causing *geometric collapse*. This distortion is independent of σ^2 and is therefore a property of the ReLU nonlinearity, not of the initialization. The empirical consequences are explored in Chapters 5 and 7.

1.3 Measuring Geometry

Previous work in this project used a PCA-based metric: $\mathrm{Var}(\mathrm{PC}_1)/\big(\mathrm{Var}(\mathrm{PC}_1) + \mathrm{Var}(\mathrm{PC}_2)\big)$, which measures the elongation of the 2D PCA projection. As we will show in Chapter 5, this metric is **misleading**: data can be spread across many dimensions (appearing "well-preserved" in PCA scatter plots) while having *no class structure* whatsoever.

We adopt three metrics that capture complementary aspects of geometry:

Definition 1.3 (k-NN accuracy)**.** Given projected data $\{\boldsymbol{h}_i^{(L)}\}_{i=1}^n$ with labels $\{y_i\}$, the k-NN accuracy is the *leave-one-out* classification accuracy using Euclidean distance with $k = 5$ neighbors.

Concretely: for each data point i, we find the $k = 5$ nearest points in the projected space (by Euclidean distance among all *other* points), and classify i by majority vote of their labels. The k-NN accuracy is the fraction of points correctly classified this way.

Chance level is $1/C$ where C is the number of classes (0.10 for Fashion-MNIST with 10 classes).

Remark 1.4 (Sensitivity to the choice of k)**.** The choice $k = 5$ is not critical. At $k = 1$, the metric is more sensitive to noise (a single nearest neighbor could be an outlier). At $k = 100$, it over-smooths and measures only coarse cluster separation. For our 2000-sample, 10-class dataset, $k = 5$ provides a stable, local measure of class structure. Empirically, results are similar for $k \in \{3, 5, 10\}$.

Definition 1.5 (Pairwise distance correlation)**.** Sample m random pairs. For each pair (i, j), compute the Euclidean distance $d_{ij}^{(0)} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|$ in the input space and $d_{ij}^{(L)} = \|\boldsymbol{h}_i^{(L)} - \boldsymbol{h}_j^{(L)}\|$ in the projected space. The metric is the Spearman rank correlation $\rho_s\big(\{d_{ij}^{(0)}\}, \{d_{ij}^{(L)}\}\big)$.

Definition 1.6 (Effective dimensionality)**.** Let $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0$ be the eigenvalues of the covariance matrix of $\{\boldsymbol{h}_i^{(L)}\}$. Define the normalized distribution $p_k = \lambda_k / \sum_j \lambda_j$. The effective dimensionality is

$$d_{\text{eff}} = \exp\Big(-\sum_k p_k \ln p_k\Big) = \exp\big(H(\{p_k\})\big), \tag{1.5}$$

where H is the Shannon entropy. This ranges from 1 (all variance in one direction) to d (uniform spread).

Remark 1.7 (Why k-NN is our primary metric)**.** The k-NN accuracy directly answers the question: *can we tell which class a point belongs to based on its neighbors in the projected space?* This is the operational definition of "class structure survives the projection."

Unlike distance correlation (which measures global distance ranking) or effective dimensionality (which measures spread), k-NN measures whether *local neighborhoods* are class-consistent — the most relevant notion for classification.

In particular, high effective dimensionality combined with low k-NN accuracy indicates *uniformly dispersed noise* (data spread across many dimensions with no class structure), as we will see for row-centered initialization in Chapter 5.

1.4 Measuring Gradient Health

1.4.1 Activation and error signal matrices

Definition 1.8 (Activation matrix)**.** Given a batch of n input samples $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$, the *activation matrix* at layer l is $A^{(l)} \in \mathbb{R}^{n \times d_l}$, where the i-th row is the post-ReLU activation vector of the i-th sample:

$$(A^{(l)})_{ik} \;=\; a_{ik}^{(l)} \;=\; [\boldsymbol{h}_i^{(l)}]_k \;=\; \mathrm{ReLU}\big(z_{ik}^{(l)}\big),$$

with pre-activation $z_{ik}^{(l)} = \sum_{j=1}^{d_{l-1}} W_{kj}^{(l)}\, a_{ij}^{(l-1)}$. In words: $a_{ik}^{(l)}$ is the value of the k-th neuron in layer l when processing sample i.

Definition 1.9 (Error signal matrix)**.** For a loss function $\mathcal{C}$, the *error signal* (backpropagated error) at layer l for sample i is

$$\boldsymbol{\delta}_i^{(l)} \;=\; \frac{\partial \mathcal{C}}{\partial \boldsymbol{z}_i^{(l)}} \;\in\; \mathbb{R}^{d_l},$$

where $\boldsymbol{z}_i^{(l)}$ is the pre-activation vector at layer l for sample i. This is computed recursively via the *backpropagation equation*:

$$\boldsymbol{\delta}_i^{(l)} = \big(W^{(l+1)}\big)^\top \boldsymbol{\delta}_i^{(l+1)} \;\odot\; \sigma'\big(\boldsymbol{z}_i^{(l)}\big), \tag{1.6}$$

where $\sigma'(\boldsymbol{z}_i^{(l)}) = \mathbf{1}[\boldsymbol{z}_i^{(l)} > 0]$ is the ReLU derivative (a binary mask: 1 where the pre-activation was positive, 0 where it was not).

The *error signal matrix* $\Delta^{(l)} \in \mathbb{R}^{n \times d_l}$ collects these row-wise: $(\Delta^{(l)})_{ik} = \delta_{ik}^{(l)}$.

1.4.2 Root-mean-square (RMS) signal levels

Using the activation matrix from Definition 1.8, we define:

$$\mathrm{rms}\big(A^{(l)}\big) = \frac{\|A^{(l)}\|_F}{\sqrt{n \cdot d_l}} = \sqrt{\frac{1}{n \cdot d_l} \sum_{i=1}^{n} \sum_{k=1}^{d_l} (a_{ik}^{(l)})^2} = \sqrt{\mathbb{E}_{i,k}\big[(a_{ik}^{(l)})^2\big]}. \tag{1.7}$$

This is the typical magnitude of a single activation entry, independent of batch size and layer width. The same definition applies to the error signal matrix: $\mathrm{rms}(\Delta^{(l)})$.

1.4.3 Per-layer gains

Definition 1.10 (Forward gain)**.**

$$g_{\mathrm{fwd}}^{(l)} = \frac{\mathrm{rms}(A^{(l)})}{\mathrm{rms}(A^{(l-1)})} = \sqrt{\frac{\mathbb{E}[(a^{(l)})^2]}{\mathbb{E}[(a^{(l-1)})^2]}}. \tag{1.8}$$

Definition 1.11 (Backward gain)**.**

$$g_{\mathrm{bwd}}^{(l)} = \frac{\mathrm{rms}(\Delta^{(l)})}{\mathrm{rms}(\Delta^{(l+1)})}, \tag{1.9}$$

using the error signal matrices from Definition 1.9. Note the direction: the backward gain measures how the error signal grows (or shrinks) as it propagates *backward* from layer $l+1$ to layer l.

1.4.4 Connection to weight gradients

The fourth backpropagation equation gives the gradient of the loss with respect to the weights at layer l. For a single sample i:

$$\frac{\partial \mathcal{C}_i}{\partial W^{(l)}} = \boldsymbol{\delta}_i^{(l)} \left(\boldsymbol{a}_i^{(l-1)}\right)^\top \in \mathbb{R}^{d_l \times d_{l-1}}.$$

Averaging over the batch of n samples and writing in matrix form:

$$\frac{\partial \mathcal{C}}{\partial W^{(l)}} = \frac{1}{n} \left(\Delta^{(l)}\right)^\top A^{(l-1)} \in \mathbb{R}^{d_l \times d_{l-1}}. \tag{1.10}$$

We can now bound the gradient magnitude. Taking the Frobenius norm:

$$\left\| \frac{\partial \mathcal{C}}{\partial W^{(l)}} \right\|_F = \frac{1}{n} \left\| (\Delta^{(l)})^\top A^{(l-1)} \right\|_F.$$

By the submultiplicativity of the Frobenius norm ($\|AB\|_F \leq \|A\|_F \, \|B\|_F$):

$$\left\| \frac{\partial \mathcal{C}}{\partial W^{(l)}} \right\|_F \leq \frac{1}{n} \|\Delta^{(l)}\|_F \cdot \|A^{(l-1)}\|_F.$$

Writing $\|\Delta^{(l)}\|_F = \mathrm{rms}(\Delta^{(l)}) \cdot \sqrt{n \cdot d_l}$ and $\|A^{(l-1)}\|_F = \mathrm{rms}(A^{(l-1)}) \cdot \sqrt{n \cdot d_{l-1}}$:

$$\left\| \frac{\partial \mathcal{C}}{\partial W^{(l)}} \right\|_F \leq \sqrt{d_l \cdot d_{l-1}} \cdot \mathrm{rms}\big(\Delta^{(l)}\big) \cdot \mathrm{rms}\big(A^{(l-1)}\big). \tag{1.11}$$

When the rows of $\Delta^{(l)}$ and $A^{(l-1)}$ are approximately uncorrelated (which holds at initialization), this bound is approximately tight. Since $\sqrt{d_l \cdot d_{l-1}}$ is the same for every layer (in a constant-width network), we conclude:

$$\left\| \frac{\partial \mathcal{C}}{\partial W^{(l)}} \right\|_F \propto \mathrm{rms}\big(A^{(l-1)}\big) \cdot \mathrm{rms}\big(\Delta^{(l)}\big). \tag{1.12}$$

The gradient magnitude at layer l is proportional to the product of the **forward signal** (activation RMS at layer $l-1$) and the **backward signal** (error signal RMS at layer l). If either varies across layers, the gradients become non-uniform.

1.4.5 Why gain ≈ 1 is critical

If the per-layer gain is a constant g across all L layers, the cumulative effect is g^L:

Per-layer gain g	g^{50}	Effect
1.01	1.64	Mild growth
1.10	1.17×10^2	Explosion
0.826	$\mathbf{7.1 \times 10^{-5}}$	**Vanishing**
0.99	0.61	Mild decay

The value $g = 0.826 = \sqrt{(\pi - 1)/\pi}$ will appear throughout this work as the structural forward gain of row-centered initialization (Chapter 3).

Chapter 2

Initialization Strategies

All initializers are defined for a weight matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ (the layout of `nn.Linear` in PyTorch, where each *row* is the weight vector of one output neuron). We write $d = d_{\text{in}}$ throughout (the number of input connections per neuron).

2.1 Baselines

These are standard initializers from the literature, providing reference points for our analysis.

2.1.1 He (Kaiming) initialization [3]

$$W_{ij} \sim \mathcal{N}\left(0,\ \frac{2}{d}\right), \qquad \text{i.e.} \quad \sigma_W = \sqrt{\frac{2}{d}}. \tag{2.1}$$

Derived by requiring $\mathbb{E}[(a^{(l)})^2] = \mathbb{E}[(a^{(l-1)})^2]$ under ReLU activation (see He et al., 2015, for the full derivation).

2.1.2 Xavier (Glorot) initialization [2]

$$W_{ij} \sim \mathcal{N}\left(0,\ \frac{2}{d_{\text{in}} + d_{\text{out}}}\right). \tag{2.2}$$

Designed for linear/sigmoid/tanh activations; does not account for ReLU's halving effect. Included as a reference baseline.

2.1.3 Uniform He

$$W_{ij} \sim \mathcal{U}(-a, a), \qquad a = \sqrt{\frac{6}{d}}, \tag{2.3}$$

so that $\text{Var}(W_{ij}) = a^2/3 = 2/d$, matching He variance with a uniform distribution.

2.1.4 Orthogonal (generic)

$$W = g \cdot Q, \qquad Q \text{ from QR decomposition of } G \sim \mathcal{N}(0,1)^{d_{\text{out}} \times d_{\text{in}}}. \tag{2.4}$$

The matrix Q is obtained from the QR decomposition of a random Gaussian matrix G, ensuring its rows form an orthonormal set. The gain parameter g scales all entries uniformly: $g = 1$ gives unit-norm rows, $g = \sqrt{2}$ gives He-scaled rows (see `orthogonal_he` in Section 2.4). This generic form is not used directly in our experiments — we use `orthogonal_he` ($g = \sqrt{2}$) instead. We define it here as the base construction.

2.2 Row-Centered Family

Row centering was proposed to combat geometric collapse: by making each neuron blind to the mean of its inputs, the hope was to reduce the DC drift that accelerates angle contraction under the arc-cosine kernel. This family explores the consequences of that constraint.

Definition 2.1 (Row centering)**.** Given a weight matrix W, the *row-centered* version is

$$\tilde{W}_{ij} = W_{ij} - \frac{1}{d}\sum_{k=1}^{d} W_{ik},$$

so that each row sums to zero: $\sum_j \tilde{W}_{ij} = 0$ for all i. Here $d = d_{\text{in}}$ is the number of input connections per neuron (i.e., the length of each row).

Motivation. When row sums are zero, the neuron output $z_i = \sum_j W_{ij} a_j$ can be rewritten as follows. Split each activation into its mean plus deviation: $a_j = \bar{a} + (a_j - \bar{a})$, where $\bar{a} = \frac{1}{d}\sum_j a_j$ is the mean activation. Then:

$$\begin{aligned} z_i &= \sum_j W_{ij}\, a_j \\ &= \sum_j W_{ij}\, \bar{a} + \sum_j W_{ij}\,(a_j - \bar{a}) \\ &= \bar{a} \underbrace{\sum_j W_{ij}}_{=0 \text{ (row centering)}} + \sum_j W_{ij}\,(a_j - \bar{a}) \\ &= \sum_j W_{ij}\,(a_j - \bar{a}). \end{aligned} \tag{2.5}$$

The key point: we are *not* adding $-\bar{a}$ by hand. We are splitting a_j into mean plus deviation, and the mean term vanishes automatically because the row sums to zero. The result is that the neuron only sees the *deviations* from the mean, not the mean itself.

2.2.1 `row_centered_he`

$$W_{ij}^{(\text{He})} \sim \mathcal{N}(0, 2/d), \qquad W_{ij} = W_{ij}^{(\text{He})} - \frac{1}{d}\sum_k W_{ik}^{(\text{He})}. \tag{2.6}$$

No variance correction. Centering reduces the per-element variance by a factor of $(1 - 1/d)$:

Proof of the variance reduction. Let $w_j \sim \mathcal{N}(0, \sigma^2)$ i.i.d. for $j = 1, \ldots, d$. After centering: $\tilde{w}_j = w_j - \bar{w}$, where $\bar{w} = \frac{1}{d}\sum_k w_k$. Then:

$$\text{Var}(\tilde{w}_j) = \text{Var}(w_j - \bar{w}) = \text{Var}(w_j) + \text{Var}(\bar{w}) - 2\,(w_j, \bar{w}).$$

Now $\text{Var}(\bar{w}) = \sigma^2/d$ (variance of the mean of d i.i.d. variables). And:

$$(w_j, \bar{w}) = \left(w_j,\ \frac{1}{d}\sum_{k=1}^{d} w_k\right) = \frac{1}{d}\,\text{Var}(w_j) = \frac{\sigma^2}{d},$$

since w_j is independent of all w_k with $k \neq j$. Therefore:

$$\text{Var}(\tilde{w}_j) = \sigma^2 + \frac{\sigma^2}{d} - 2\,\frac{\sigma^2}{d} = \sigma^2\left(1 - \frac{1}{d}\right). \qquad \square$$

Geometrically, centering projects each row onto the hyperplane orthogonal to $\mathbf{1}$, removing one degree of freedom out of d.

2.2.2 `row_centered_he_var_adj`

1. Sample $W^{(\text{He})} \sim \mathcal{N}(0, 2/d)$.
2. Center each row: $\tilde{\boldsymbol{w}}_i = \boldsymbol{w}_i - \bar{w}_i \mathbf{1}$.
3. Rescale each row to restore He variance: $\boldsymbol{w}_i \leftarrow \tilde{\boldsymbol{w}}_i \cdot \sqrt{2/d} \,/\, \text{std}(\tilde{\boldsymbol{w}}_i)$.

Why the rescaling (step 3). After centering, each row has per-element standard deviation $\approx \sigma_W \sqrt{1 - 1/d}$ (from the variance reduction above). For $d = 784$, this is $\sigma_W \cdot 0.9994$ — nearly identical, but the rescaling ensures *exactly* $\text{Var}(W_{ij}) = 2/d$ (matching He). Each row is divided by its empirical standard deviation and multiplied by $\sqrt{2/d}$. The result: centered rows (sum to zero) with exactly He variance.

2.2.3 `row_centered_forward_balanced` (diagnostic)

Motivation. In Chapter 3, we show that row centering causes a forward gain of $\sqrt{(\pi-1)/\pi} \approx 0.826$ per layer. A natural question: *can we simply increase the weight variance to compensate?* This initializer answers that question.

Derivation. For a row-centered layer with variance $\text{Var}(W_{ij}) = v$, the forward gain is (from Chapter 3):

$$g_{\text{fwd}} = \sqrt{v \cdot \frac{d}{2} \cdot \frac{\pi - 1}{\pi}}.$$

Setting $g_{\text{fwd}} = 1$ and solving for v:

$$v \cdot \frac{d}{2} \cdot \frac{\pi - 1}{\pi} = 1 \quad \Longrightarrow \quad v = \frac{2\pi}{(\pi - 1)\, d} \approx \frac{2.934}{d}.$$

So the required weight variance is:

$$\text{Var}(W_{ij}) = \frac{2\pi}{(\pi - 1)\, d}. \tag{2.7}$$

Result. This is a **diagnostic** initializer: it proves that forcing the forward gain to 1.0 pushes the backward gain to $\sqrt{\pi/(\pi-1)} \approx 1.21$, causing backward explosion ($1.21^{50} \approx 1.3 \times 10^4$). You cannot fix one direction without breaking the other (Section 3.5).

2.2.4 `row_centered_layer_balanced`

Motivation. Instead of using the same weight variance at every layer, we use a *per-layer variance schedule* σ_l that partially compensates for the cumulative forward decay.

Definition.

$$\sigma_l = \sigma_{\text{base}} \cdot r^{\,\eta\,(l - (L+1)/2)}, \qquad \sigma_{\text{base}} = \sqrt{\frac{2}{d}} \cdot \sqrt{\frac{\pi}{\pi - 1}}, \tag{2.8}$$

where $r = \sqrt{(\pi-1)/\pi}$, $l \in \{1, \ldots, L\}$ is the layer index, L is the total number of layers, and $\eta \in [0, 1]$ controls the correction strength.

Each layer's weight matrix is sampled $W \sim \mathcal{N}(0, \sigma_l)$, then row-centered and rescaled to have per-row standard deviation σ_l.

Intuition. The parameter η controls how much per-layer compensation to apply:

- $\eta = 0$: uniform variance across layers, same as `forward_balanced` (forward gain ≈ 1.0, backward gain ≈ 1.21 at every layer).
- $\eta = 1$: full per-layer correction, with early layers getting smaller variance and later layers getting larger variance to fight cumulative decay.
- Intermediate η: a partial correction.

Flag. The base standard deviation σ_{base} includes the $\sqrt{\pi/(\pi-1)}$ factor, which is the **forward-balanced** correction (Section 2.2.3), not the variance-adjusted base. Consequently, at $\eta = 0$ this initializer gives:

- Forward gain ≈ 1.0 (not 0.826)
- Backward gain ≈ 1.21 (not 1.0)

This differs from `row_centered_he_var_adj`, which at the same depth would give forward gain ≈ 0.826 and backward gain ≈ 1.0. The distinction matters when interpreting the η-sweep results in Section 4.2.

The full mathematical derivation of why this scheme partially equalizes gradients is given in Section 4.2.

2.3 Partial Centering

Since full row centering creates a gradient trap (Chapter 3), a natural follow-up: *what if we only partially center?* This family interpolates between He (no centering, good gradients) and full row centering (geometry-motivated, but trapped gradients).

2.3.1 `partial_centered_he`

$$W_{ij} = W_{ij}^{(\text{He})} - \alpha \cdot \frac{1}{d} \sum_k W_{ik}^{(\text{He})}, \tag{2.9}$$

followed by rescaling each row to restore He variance.

The parameter $\alpha \in [0, 1]$ interpolates between pure He ($\alpha = 0$, no centering) and full row centering ($\alpha = 1$). At intermediate α, the row sums are reduced but not zero: $\sum_j W_{ij} = (1-\alpha) \sum_j W_{ij}^{(\text{He})}$.

Default: $\alpha = 0.5$.

2.4 Orthogonal Family

Orthogonal initialization preserves norms exactly in the linear part of the forward pass (before the nonlinearity). Combined with He scaling, it provides the benefits of structured weights without the zero-sum constraint that causes the gradient trap.

2.4.1 `orthogonal_he`

$$W = \sqrt{2} \cdot Q, \qquad Q \text{ orthogonal from QR.} \tag{2.10}$$

The gain $\sqrt{2}$ matches He scaling: since ReLU zeroes $\sim$50% of outputs, the factor of 2 compensates for the halving. Orthogonal rows are linearly independent but do **not** have the zero-sum constraint, so there is no gradient trap.

2.4.2 `orthogonal_tuned`

$$W = \sqrt{1.65} \cdot Q. \tag{2.11}$$

Uses the tuned factor 1.65 instead of 2.

In our experiments (Section 5.3), `orthogonal_tuned` produces **identical results** to `orthogonal_he` on all scale-invariant metrics (k-NN accuracy, distance correlation). This is because for square

matrices with the same random seed, the QR decomposition produces the same orthogonal matrix Q; only the scalar gain differs, which does not affect angle-based or rank-based metrics. We include it for completeness but focus on `orthogonal_he` in our analysis.

2.5 Experimental Initializers

These are exploratory approaches that test specific hypotheses about fixing the gradient trap or the kernel distortion.

2.5.1 `centered_with_dc_he`

1. Sample and center as in `row_centered_he_var_adj`.
2. Add a constant DC offset to every element:

$$W_{ij} \leftarrow W_{ij} + \delta \cdot \sqrt{2/d}, \qquad \delta = 0.1 \text{ (default)}.$$

This breaks the zero-sum constraint ($\sum_j W_{ij} = d \cdot \delta\sqrt{2/d} \neq 0$) while preserving most of the centering benefit.

2.5.2 `kernel_preserving`

This initializer directly minimizes the kernel distortion via optimization. Starting from He initialization, it runs 200 Adam steps (learning rate 0.01) on the objective:

$$\min_W \sum_{i,j} \left(\frac{\langle \mathrm{ReLU}(W\boldsymbol{x}_i), \mathrm{ReLU}(W\boldsymbol{x}_j) \rangle}{d_{\text{out}}} - \langle \boldsymbol{x}_i, \boldsymbol{x}_j \rangle \right)^2 + \lambda \sum_i \left(\frac{\| \mathrm{ReLU}(W\boldsymbol{x}_i) \|^2}{d_{\text{out}}} - 1 \right)^2, \quad (2.12)$$

where $\{\boldsymbol{x}_i\}$ are 500 random unit vectors (data-independent proxy) and $\lambda = 1.0$.

Flag. Normalization artifact. Both terms in (2.12) are divided by d_{out}. The norm constraint therefore asks for $\| \mathrm{ReLU}(W\boldsymbol{x}) \|^2 / d_{\text{out}} \approx 1$, i.e., the optimizer targets output vectors of length $\| \mathrm{ReLU}(W\boldsymbol{x}) \| \approx \sqrt{d_{\text{out}}}$. For our layer width $d = 784$, this means $\sqrt{784} = 28$, not 1.

When we later observe (Section 6.2) that the optimized weights are "11.5× inflated" compared to He, part of this inflation is simply because the optimizer is targeting larger output norms than He does — it is partly a normalization artifact, not necessarily a meaningful structural difference.

2.5.3 `custom_variance`

$$W_{ij} \sim \mathcal{N}(\mu, \sigma^2), \quad (2.13)$$

with user-specified μ (default 0) and σ^2 (default $2/d$). Used for variance sweep experiments.

2.6 Summary Table

Remark 2.2 (Key initializers for discussion)**.** The most important rows in this table are:

- `he`: the standard baseline — good gradients, reasonable geometry.
- `rc_he_var_adj`: the professor's proposal with variance correction — this is what we analyze in depth.

Initializer	Row sum = 0?	Var adjusted?	Gradient trap?	Best for
`he`	No	N/A	No	Gradient baseline
`xavier`	No	N/A	No	Linear/tanh
`uniform_he`	No	N/A	No	Uniform baseline
`row_centered_he`	Yes	No	Yes	Geometry baseline
`rc_he_var_adj`	Yes	Yes	Yes	Geom. + correct var
`rc_fwd_balanced`	Yes	Yes (2.93)	Yes	Diagnostic only
`rc_layer_balanced`	Yes	Per-layer	Yes	Geom. + grad. balance
`partial_centered_he`	Partial	Yes	Partial	Trade-off
`orthogonal_he`	No	N/A	No	Geom. without trap
`orthogonal_tuned`	No	N/A	No	≡ orthogonal_he
`centered_with_dc`	Nearly	Yes	Partial	Breaking the trap
`kernel_preserving`	No	Optimized	No	Shallow geom.
`custom_variance`	No	Custom	No	Experiments

Table 2.1: Summary of all initialization strategies.

- `rc_fwd_balanced`: diagnostic variant that reveals the gradient trap coupling (Section 3.5).
- `orthogonal_he`: the practical winner — best geometry with no gradient trap (Section 5.3).
- `kernel_preserving`: shows that direct optimization finds the *opposite* of centering (Section 6.2).

The remaining initializers are supporting experiments that help us understand the landscape of possible solutions.

Chapter 3

The Gradient Trap: Forward-Backward Asymmetry

Experimental setup. 50 hidden layers, width 784, Fashion-MNIST (2000 samples). We compare three initializers: He (baseline), `row_centered_he_var_adj`, and `row_centered_forward_balanced`.

3.1 The Centering Ratio: $(\pi - 1)/\pi$

3.1.1 Half-Gaussian moments

After ReLU, the activations follow a half-Gaussian distribution. If the pre-activation $z \sim \mathcal{N}(0, \sigma^2)$, then $a = \mathrm{ReLU}(z) = \max(0, z)$ satisfies:

Lemma 3.1 (Half-Gaussian moments)**.**

$$\mathbb{E}[a] = \frac{\sigma}{\sqrt{2\pi}}, \qquad \mathbb{E}[a^2] = \frac{\sigma^2}{2}. \tag{3.1}$$

Proof. Since $\Pr(z > 0) = 1/2$ and the positive half of $\mathcal{N}(0, \sigma^2)$ is a half-normal distribution:

$$\mathbb{E}[a] = \mathbb{E}[z \mid z > 0] \cdot \Pr(z > 0) = \frac{\sigma\sqrt{2/\pi}}{1} \cdot \frac{1}{2} = \frac{\sigma}{\sqrt{2\pi}}.$$

For the second moment: $\mathbb{E}[a^2] = \mathbb{E}[z^2 \mid z > 0] \cdot \Pr(z > 0) = \sigma^2 \cdot \frac{1}{2}$. □

From these moments, the variance of a post-ReLU activation is:

$$\mathrm{Var}(a) = \mathbb{E}[a^2] - (\mathbb{E}[a])^2 = \frac{\sigma^2}{2} - \frac{\sigma^2}{2\pi} = \frac{\sigma^2}{2}\left(1 - \frac{1}{\pi}\right) = \frac{\sigma^2(\pi - 1)}{2\pi}. \tag{3.2}$$

3.1.2 Why the centering ratio matters

Before stating the centering ratio formally, let us explain *why* this quantity is the key to understanding the gradient trap.

Post-ReLU activations are always non-negative ($a_j \geq 0$), so they have a positive mean $\mathbb{E}[a] > 0$. This mean is a *constant offset* — it depends on the weight variance σ^2 but not on which input $\boldsymbol{x}$ was fed in. It carries no information about the class of the input.

Recall from equation (2.5) that row-centered weights ($\sum_j W_{ij} = 0$) are blind to any constant component in their input. So row-centered weights can only access the *varying* part of the activations — the variance $\mathrm{Var}(a)$.

The centering ratio $\mathrm{Var}(a)/\mathbb{E}[a^2]$ answers a precise question: *what fraction of the total activation energy is accessible to row-centered weights?*

Why $\mathbb{E}[a^2]$ *and not* $\mathbb{E}[a]$*?* Signal propagation depends on *energy* (second moments), not first moments. The forward gain (Definition 1.8) compares $\mathbb{E}[(a^{(l)})^2]$ across layers, so the relevant question is what fraction of *energy* is available, not what fraction of the *mean* is available.

Proposition 3.2 (Centering ratio). *For post-ReLU activations with pre-activation* $z \sim \mathcal{N}(0, \sigma^2)$*:*

$$\frac{\mathrm{Var}(a)}{\mathbb{E}[a^2]} = \frac{\sigma^2(\pi - 1)/(2\pi)}{\sigma^2/2} = \frac{\pi - 1}{\pi} \approx 0.6817. \tag{3.3}$$

This ratio is ***independent of*** σ *— it is a universal property of the ReLU activation function.*

Interpretation. Out of the total signal energy $\mathbb{E}[a^2]$, only 68.2% is accessible to row-centered weights. The remaining 31.8% is the constant DC offset that gets ignored.

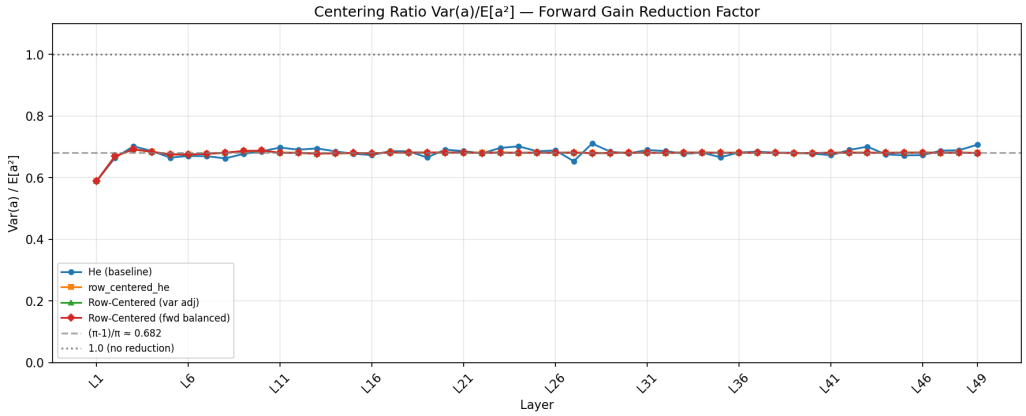

Figure 3.1: Empirical centering ratio $\mathrm{Var}(a)/\mathbb{E}[a^2]$ per layer. All four initializers converge to $(\pi - 1)/\pi \approx 0.682$, confirming this is a universal property of post-ReLU activations. Source: `notebooks/06_gradient_diagnostics.ipynb`, Cell 10.

3.1.3 DC/AC decomposition

A post-ReLU activation vector at layer l has d components (one per neuron in that layer): $\boldsymbol{a}^{(l)} = (a_1, \ldots, a_d) \in \mathbb{R}^d$. Each component can be decomposed into its mean plus deviation:

$$a_j = \underbrace{\bar{a}}_{\text{DC (mean)}} + \underbrace{(a_j - \bar{a})}_{\text{AC (deviation)}}, \qquad \bar{a} = \frac{1}{d}\sum_{j=1}^{d} a_j. \tag{3.4}$$

What is "energy"? In signal processing, the *energy* of a signal $\{a_1, \ldots, a_d\}$ is $\sum_j a_j^2$. The average per-element energy is $\frac{1}{d}\sum_j a_j^2 \approx \mathbb{E}[a^2]$ (by the law of large numbers when d is large). This second moment measures how large the activations are on average, regardless of sign.

Why $\mathbb{E}[a^2]$ splits into DC + AC. From the standard variance decomposition:

$$\mathbb{E}[a^2] = \underbrace{(\mathbb{E}[a])^2}_{\text{DC energy}} + \underbrace{\mathrm{Var}(a)}_{\text{AC energy}}.$$

- **DC energy** $(\mathbb{E}[a])^2 = \sigma^2/(2\pi)$: This is the energy of the *constant* (mean) component. By analogy with electrical signals, this is the "DC" (direct current) component — a constant offset that is the *same for all inputs*. Whether we feed in a shoe, a shirt, or a bag, the mean activation $\mathbb{E}[a] = \sigma/\sqrt{2\pi}$ is always the same. It carries **zero information** about the class.

- **AC energy** $\mathrm{Var}(a) = \sigma^2(\pi-1)/(2\pi)$: This is the energy of the *fluctuating* component — the "AC" (alternating current) part that varies from input to input. The deviations $(a_j - \bar{a})$ are different for different inputs, so they preserve the structure of the data. This is the **informative** part.

The fraction of energy that is informative: AC/total $= (\pi-1)/\pi \approx 68.2\%$. The remaining $1/\pi \approx 31.8\%$ is uninformative DC — a constant offset present in every activation vector, useless for distinguishing classes.

3.1.4 Why row-centered weights are blind to DC

Proposition 3.3 (DC blindness). *If $\sum_j W_{ij} = 0$ (row centering), then*

$$z_i = \sum_j W_{ij}\, a_j = \sum_j W_{ij}\, (a_j - \bar{a}).$$

The neuron's output depends only on the AC component of its input.

Proof. We split each activation into its mean plus deviation. When row-centered weights ($\sum_j W_{ij} = 0$) multiply the mean part, it vanishes:

$$\sum_j W_{ij}\, a_j = \sum_j W_{ij}\, \bar{a} + \sum_j W_{ij}\, (a_j - \bar{a}) = \bar{a} \underbrace{\sum_j W_{ij}}_{=0} + \sum_j W_{ij}\, (a_j - \bar{a}) = \sum_j W_{ij}\, (a_j - \bar{a}). \quad \square$$

Consequence for signal propagation. When standard He weights compute $z_i = \sum_j W_{ij}\, a_j$, the weights see the full activation vector including its positive mean. The variance of the pre-activation is:

$$\mathrm{Var}(z_i) = d \cdot \mathrm{Var}(W) \cdot \mathbb{E}[a^2] \quad \text{— uses the } \textit{full} \text{ second moment } \mathbb{E}[a^2].$$

When row-centered weights compute the same sum, the DC part vanishes (Proposition 3.3), so effectively $z_i = \sum_j W_{ij}(a_j - \bar{a})$. The variance becomes:

$$\mathrm{Var}(z_i) = d \cdot \mathrm{Var}(W) \cdot \mathrm{Var}(a) = d \cdot \mathrm{Var}(W) \cdot \frac{\pi-1}{\pi}\, \mathbb{E}[a^2].$$

The factor $(\pi-1)/\pi \approx 0.682$ means row-centered weights produce pre-activations with only 68.2% of the energy that He weights produce. This is the energy deficit that compounds across layers.

3.2 Forward Gain Derivation

3.2.1 Standard He: forward gain $= 1$

For a He-initialized layer (no centering), the expected output energy is:

$$\mathbb{E}[(a^{(l)})^2] = \underbrace{\mathrm{Var}(W) \cdot d}_{=(2/d)\cdot d = 2} \cdot \underbrace{\frac{1}{2}}_{\text{ReLU survival}} \cdot \mathbb{E}[(a^{(l-1)})^2] = \mathbb{E}[(a^{(l-1)})^2]. \tag{3.5}$$

The forward gain is $g_{\text{fwd}}^{\text{He}} = \sqrt{\mathbb{E}[(a^{(l)})^2]/\mathbb{E}[(a^{(l-1)})^2]} = 1$.

3.2.2 Row-centered: forward gain $= \sqrt{(\pi-1)/\pi}$

The key step: in the He derivation (3.5), the factor $\mathbb{E}[(a^{(l-1)})^2]$ appears because He weights see the *full* activation energy. For row-centered weights, we must replace this with $\mathrm{Var}(a^{(l-1)})$ (because of DC blindness, Proposition 3.3). Since $\mathrm{Var}(a) = \frac{\pi-1}{\pi}\,\mathbb{E}[a^2]$ (Proposition 3.2), we get the extra factor $(\pi-1)/\pi$.

For a row-centered layer with variance-adjusted weights ($\mathrm{Var}(W_{ij}) = 2/d$):

$$\mathbb{E}[(a^{(l)})^2] = \underbrace{\mathrm{Var}(W)\cdot d\cdot\frac{1}{2}}_{=1}\cdot\underbrace{\mathrm{Var}(a^{(l-1)})}_{\text{only AC visible}} = \mathrm{Var}(a^{(l-1)}) = \frac{\pi-1}{\pi}\,\mathbb{E}[(a^{(l-1)})^2]. \tag{3.6}$$

This gives a multiplicative recurrence in the energy:

$$\mathbb{E}[(a^{(l)})^2] = \frac{\pi-1}{\pi}\,\mathbb{E}[(a^{(l-1)})^2].$$

At every layer, row-centered weights produce activations whose energy is $(\pi-1)/\pi \approx 0.682$ times the energy of the previous layer's activations. The signal shrinks at every layer.

The forward gain is the ratio of **amplitudes** (RMS), not energies:

$$\boxed{g_{\text{fwd}}^{\text{RC}} = \sqrt{\frac{\mathbb{E}[(a^{(l)})^2]}{\mathbb{E}[(a^{(l-1)})^2]}} = \sqrt{\frac{\pi-1}{\pi}} \approx 0.826.} \tag{3.7}$$

Remark 3.4 (Amplitude vs. energy ratio)**.** The forward gain is defined as $g_{\text{fwd}} = \mathrm{rms}(A^{(l)})/\,\mathrm{rms}(A^{(l-1)})$, which is a ratio of *amplitudes* (RMS values). Since $\mathrm{rms}^2 \propto \mathbb{E}[a^2]$ (energy), we have

$$g_{\text{fwd}}^2 = \frac{\mathbb{E}[(a^{(l)})^2]}{\mathbb{E}[(a^{(l-1)})^2]} = \frac{\pi-1}{\pi} \approx 0.682.$$

Taking the square root: $g_{\text{fwd}} = \sqrt{0.682} \approx 0.826$.

Analogy: if you halve the power of a light bulb (energy ratio $= 1/2$), the brightness (amplitude) drops by $\sqrt{1/2} \approx 0.707$, not by $1/2$. Similarly, an energy ratio of 0.682 corresponds to an amplitude ratio of $\sqrt{0.682} = 0.826$.

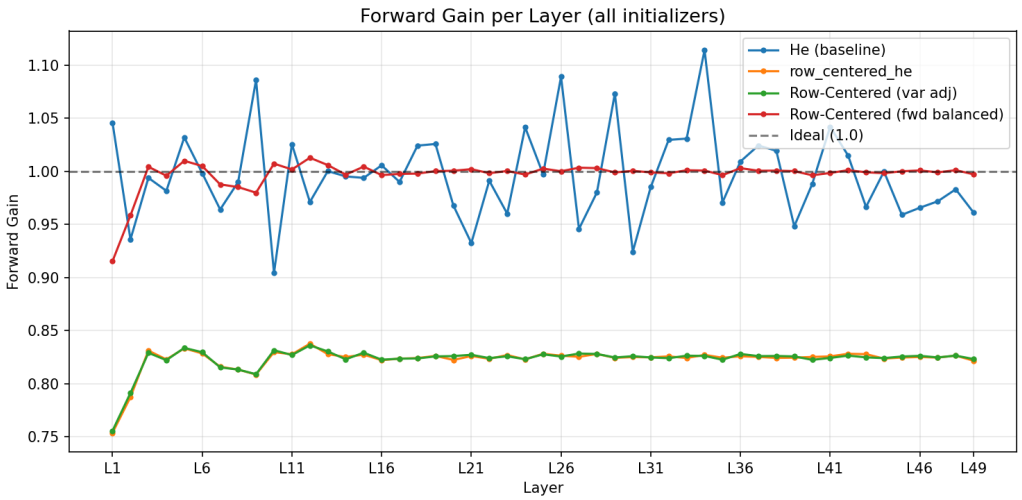

Figure 3.2: Forward gain per layer for four initializers (50 hidden layers, width 784, Fashion-MNIST). He and RC fwd-balanced achieve gain ≈ 1.0; RC var-adj and `row_centered_he` are locked at ≈ 0.826. Source: `notebooks/06_gradient_diagnostics.ipynb`, Cell 8.

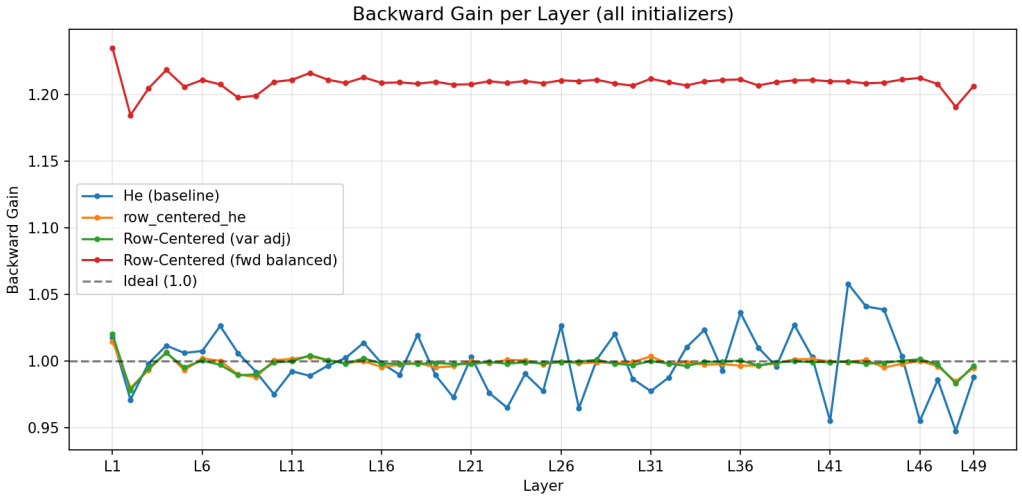

Figure 3.3: Backward gain per layer. He, RC var-adj, and `row_centered_he` all have gain ≈ 1.0. RC fwd-balanced has gain ≈ 1.21 — error signals explode at $1.21^{50} \approx 1.3 \times 10^4$. Source: `notebooks/06_gradient_diagnostics.ipynb`, Cell 8.

3.2.3 Compounding over depth

Over L layers:

$$\text{rms}(a^{(L)}) = \left(\frac{\pi - 1}{\pi}\right)^{L/2} \text{rms}(a^{(0)}) \approx 0.826^L \cdot \text{rms}(a^{(0)}). \tag{3.8}$$

Depth L	Forward attenuation 0.826^L	
10	0.147	(6.8× decay)
20	0.022	(46× decay)
50	7.1×10^{-5}	(14,000× decay)

By layer 50, the activations are 14,000 times smaller than the input. From (1.11), the gradient at layer l is proportional to $\|\boldsymbol{a}^{(l-1)}\|$, so later layers receive vanishingly small gradients.

3.3 Why the Backward Gain Is Unaffected

The backward pass propagates the error signal via:

$$\boldsymbol{\delta}^{(l)} = \left(W^{(l+1)}\right)^\top \boldsymbol{\delta}^{(l+1)} \odot \sigma'(\boldsymbol{z}^{(l)}), \tag{3.9}$$

where $\sigma'(\boldsymbol{z}^{(l)}) = \mathbf{1}[\boldsymbol{z}^{(l)} > 0]$ is the ReLU derivative (binary mask).

If W is row-centered ($\sum_j W_{ij} = 0$), then $W^\top$ is *column-centered* ($\sum_i W_{ij} = 0$ for each column j). Applying $W^\top$ to $\boldsymbol{\delta}^{(l+1)}$ gives:

$$(W^\top \boldsymbol{\delta})_j = \sum_i W_{ij}\, \delta_i.$$

The centering of columns means this is blind to the mean of $\boldsymbol{\delta}^{(l+1)}$. However, unlike post-ReLU activations, the error signal $\boldsymbol{\delta}$ has **near-zero mean** (it contains both positive and negative values). Therefore:

$$\frac{\text{Var}(\delta)}{\mathbb{E}[\delta^2]} = 1 - \frac{(\mathbb{E}[\delta])^2}{\mathbb{E}[\delta^2]} \approx 1,$$

and the column centering of $W^\top$ has negligible effect on the backward signal energy.

Proposition 3.5 (Forward-backward gain ratio)**.** *For row-centered initialization with any weight variance:*

$$\frac{g_{\text{fwd}}}{g_{\text{bwd}}} = \sqrt{\frac{\pi - 1}{\pi}} \approx 0.827. \tag{3.10}$$

This ratio is ***independent of the weight variance****.*

This is the fundamental coupling: the forward and backward gains are locked in a fixed ratio. No scalar variance adjustment can make both equal to 1 simultaneously.

3.4 Empirical Confirmation: Variance Sweep

We sweep the variance factor for row-centered initialization from $1.5/d$ to $3.0/d$:

Variance factor	Median fwd gain	Median bwd gain	Ratio F/B
$1.5/d$	0.715	0.865	**0.827**
$2.0/d$ (He)	0.826	0.999	**0.827**
$2.5/d$	0.923	1.116	**0.827**
$3.0/d$	1.011	1.223	**0.827**

The ratio $g_{\text{fwd}}/g_{\text{bwd}} = 0.827$ is constant across all variance factors, confirming Proposition 3.5.
Interpretation:

- At Var $= 2.0/d$: backward gain $= 1.0$ (healthy), but forward gain $= 0.826$ (activations vanish).
- At Var $\approx 2.93/d$: forward gain $= 1.0$ (healthy), but backward gain $= 1.21$ (error signals explode: $1.21^{50} \approx 1.3 \times 10^4$).
- No variance gives both $= 1$.

3.5 The Coupling Formula

Writing the forward gain as $g_{\text{fwd}} = r \cdot s$ where $r = \sqrt{(\pi - 1)/\pi} \approx 0.826$ is the structural centering factor and s depends on the weight variance, and the backward gain as $g_{\text{bwd}} = s$:

$$\frac{g_{\text{fwd}}}{g_{\text{bwd}}} = r \approx 0.826 \quad \text{always.} \tag{3.11}$$

- $s = 1$ (He variance, `var_adj`): $g_{\text{fwd}} = 0.826$, $g_{\text{bwd}} = 1.0 \Rightarrow$ activations vanish.
- $s = 1/r \approx 1.21$ (`fwd_balanced`): $g_{\text{fwd}} = 1.0$, $g_{\text{bwd}} = 1.21 \Rightarrow$ error signals explode.
- Any s in between: both are off from 1.

This is why geometry (requiring row centering) and gradient stability (requiring gain ≈ 1) conflict.

3.6 Single vs. Multi-Sample: The Trap Is Structural

To verify that the gradient non-uniformity is not a statistical artifact of batch aggregation, we compare gradient profiles using $n = 1$ vs. $n = 2000$ samples.

Initializer	Log-scale correlation	Interpretation
He	0.86	Noisier (no centering stabilization)
RC var-adj	0.98	Same pattern
RC fwd-balanced	0.98	Same pattern

The row-centered variants show correlation > 0.97 between single-sample and batch gradient profiles. The gradient decay pattern is identical whether we use one data point or two thousand.

Conclusion: The gradient non-uniformity is **structural** — it comes from the weight matrices and the ReLU activation, not from data aggregation.

3.7 ReLU Survival Is Not the Issue

All three initializers show ReLU survival rates of approximately $50\% \pm 2\%$ across all layers:

- He: $50\% \pm 1.7\%$ (higher noise)
- RC var-adj: $50\% \pm 0.4\%$ (tighter due to centering)
- RC fwd-balanced: $50\% \pm 0.4\%$

Row centering does **not** kill more neurons than standard He. The gradient trap is not about dead neurons — it is about the *positive mean* of surviving activations (the DC component that row-centered weights cannot see).

Chapter 4

Attempting to Fix the Trap

Chapter 3 established that row centering creates a locked forward/backward gain ratio of $\sqrt{(\pi-1)/\pi} \approx 0.827$. This chapter explores two strategies to mitigate the resulting gradient non-uniformity.

4.1 Partial Centering (Alpha Sweep)

4.1.1 Definition

For $\alpha \in [0, 1]$, define partial centering:

$$W_{ij} = W_{ij}^{(\mathrm{He})} - \alpha \cdot \bar{w}_i, \qquad \bar{w}_i = \frac{1}{d}\sum_k W_{ik}^{(\mathrm{He})}, \tag{4.1}$$

followed by rescaling each row to restore He variance. At $\alpha = 0$ this is pure He; at $\alpha = 1$ this is full row centering.

4.1.2 Effect on forward gain

Partial centering with parameter α suppresses a fraction of the DC component. The effective input energy seen by the neuron is approximately:

$$E_{\mathrm{eff}} \approx \mathbb{E}[a^2] - \alpha^2 \cdot (\mathbb{E}[a])^2 = \mathbb{E}[a^2]\left(1 - \frac{\alpha^2}{\pi}\right), \tag{4.2}$$

giving forward gain approximately:

$$g_{\mathrm{fwd}}(\alpha) \approx \sqrt{1 - \frac{\alpha^2}{\pi}} \approx 1 - \frac{\alpha^2}{2\pi}. \tag{4.3}$$

A simpler linear approximation from the data: $g_{\mathrm{fwd}}(\alpha) \approx 1 - 0.174\,\alpha$.

4.1.3 Empirical results

Setup: 50 hidden layers, width 784, Fashion-MNIST. PCA geometry metric (used before the k-NN transition; see Chapter 5 for why this is misleading).

α	Med fwd gain	Med bwd gain	Grad spread	Geometry (PCA)
0.0	0.996	0.997	6.6×	0.955
0.2	0.937	0.996	2.4×	0.905
0.4	0.887	0.999	36.8×	0.835
0.6	0.853	0.998	294.7×	0.822
0.8	0.832	0.999	1,219×	0.808
1.0	0.826	0.999	1,908×	0.818

Key observations:

1. **Backward gain ≈ 1.0 for all α:** The backward pass is robust to partial centering because the transposed weights interact with error signals that have near-zero mean.

2. **Gradient spread grows exponentially with α:** Even $\alpha = 0.5$ gives $g_{\text{fwd}} \approx 0.913$, which over 50 layers compounds to $0.913^{50} \approx 0.010$ — three orders of magnitude decay.

3. **No α gives both good gradients and good geometry:** $\alpha = 0.2$ minimizes gradient spread (2.4×) but has poor geometry. $\alpha \geq 0.6$ gives good geometry but gradient spreads $> 300\times$.

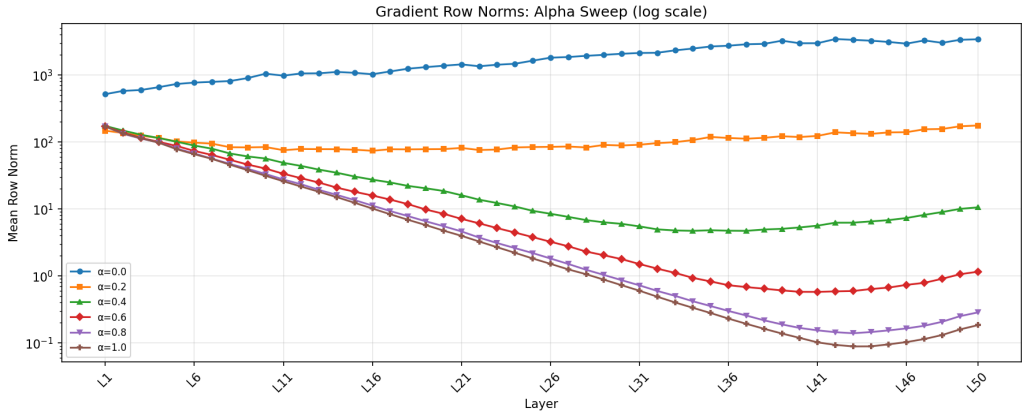

Figure 4.1: Gradient row norms (log scale) across 50 layers for partial centering with $\alpha \in \{0, 0.2, 0.4, 0.6, 0.8, 1.0\}$. At $\alpha = 0$ (He), gradients are nearly flat. At $\alpha = 1$ (full centering), gradients span $\sim$4 orders of magnitude. Source: `notebooks/06_gradient_diagnostics.ipynb`, Cell 20.

4.1.4 Conclusion

Partial centering defines a **Pareto frontier** between geometry and gradient uniformity. It allows choosing *where* on the trade-off to sit, but cannot escape it. Any α large enough to meaningfully suppress DC drift creates enough forward signal loss to produce non-uniform gradients.

4.2 Layer-Balanced Initialization (Eta Sweep)

4.2.1 Motivation

The partial centering approach tried to reduce the centering parameter. The layer-balanced approach takes a fundamentally different strategy: **keep full row centering ($\alpha = 1$) for best geometry, but use different weight variances at each layer to equalize gradients despite the forward decay.**

4.2.2 Mathematical derivation

Let each layer $l \in \{1, \ldots, L\}$ have weight standard deviation s_l. From the backpropagation equations:

Forward chain (activation magnitude at layer $l-1$):

$$\|\boldsymbol{a}^{(l-1)}\| \propto \prod_{k=1}^{l-1} (r \cdot s_k), \tag{4.4}$$

where $r = \sqrt{(\pi-1)/\pi} \approx 0.826$ is the structural centering loss per layer, and $r \cdot s_k$ is the forward amplitude gain at layer k.

Backward chain (error signal at layer l):

$$\|\boldsymbol{\delta}^{(l)}\| \propto \prod_{k=l+1}^{L} s_k. \tag{4.5}$$

The backward gain at layer k is approximately s_k (Section 3.3).

Gradient at layer l (from BP4, equation (1.10)):

$$\left\| \frac{\partial \mathcal{C}}{\partial W^{(l)}} \right\| \propto \|\boldsymbol{a}^{(l-1)}\| \cdot \|\boldsymbol{\delta}^{(l)}\| \propto \prod_{k=1}^{l-1}(r \cdot s_k) \cdot \prod_{k=l+1}^{L} s_k = r^{l-1} \cdot \frac{\prod_{k=1}^{L} s_k}{s_l}. \tag{4.6}$$

Since $\prod_{k=1}^{L} s_k$ is a constant (independent of l), the gradient depends on l only through:

$$\left\| \frac{\partial \mathcal{C}}{\partial W^{(l)}} \right\| \propto \frac{r^{l-1}}{s_l}. \tag{4.7}$$

For uniform gradients, we need $r^{l-1}/s_l = \text{const}$, which gives:

$$s_l \propto r^{l-1}. \tag{4.8}$$

Early layers (l small) get s_l large (boosted weights); late layers (l large) get s_l small (reduced weights).

4.2.3 Parameterization

We center the scaling at the middle layer and introduce a strength parameter η:

$$s_l = r^{\,\eta \cdot (l-(L+1)/2)}, \tag{4.9}$$

so that $s_{(L+1)/2} = 1$ (middle layer unchanged).

- $\eta = 0$: $s_l = 1$ for all l (no correction).
- $\eta = 1$: full correction ($s_l = r^{l-(L+1)/2}$).
- $\eta = 0.5$: half correction.

The total weight std at layer l is $\sigma_l = \sigma_{\text{base}} \cdot s_l$, where $\sigma_{\text{base}} = \sqrt{2/d} \cdot \sqrt{\pi/(\pi-1)}$ is the forward-balanced base (see the flag in Section 2.2.4).

4.2.4 Results

Setup: 20 hidden layers, width 784, Fashion-MNIST.

Config	Grad ratio	Grad CV	Med fwd gain	Med bwd gain
He (baseline)	1.93×	0.182	0.979	1.022
$\eta = 0.00$	11.41×	0.926	0.999	1.213
$\eta = 0.25$	6.52×	0.603	1.045	1.212
$\boldsymbol{\eta = 0.50}$	**4.32×**	**0.521**	1.096	1.212
$\eta = 0.75$	6.67×	0.764	1.150	1.212
$\eta = 1.00$	12.93×	1.075	1.206	1.212

Flag. At $\eta = 0$, the median forward gain is 0.999 and the backward gain is 1.213. This means $\eta = 0$ corresponds to the **forward-balanced** base (Section 2.2.3), *not* the variance-adjusted base (which would give forward gain ≈ 0.826 and backward gain ≈ 1.0).

4.2.5 Analysis

1. $\eta = 0.5$ **is the sweet spot:** Gradient ratio improves from 11.41× to 4.32× ($\sim$ 2.6× improvement), CV from 0.926 to 0.521. Still worse than He's 1.93×.

2. $\eta = 1.0$ **is worse than** $\eta = 0$**:** Gradient ratio 12.93× vs. 11.41×. The full correction **overshoots**.

3. **Backward gain is** 1.21 **for ALL** η**:** The backward gain depends on the average weight scale (the forward-balanced base), not on how variance is distributed across layers. Over 20 layers: $1.21^{20} \approx 55\times$ backward amplification.

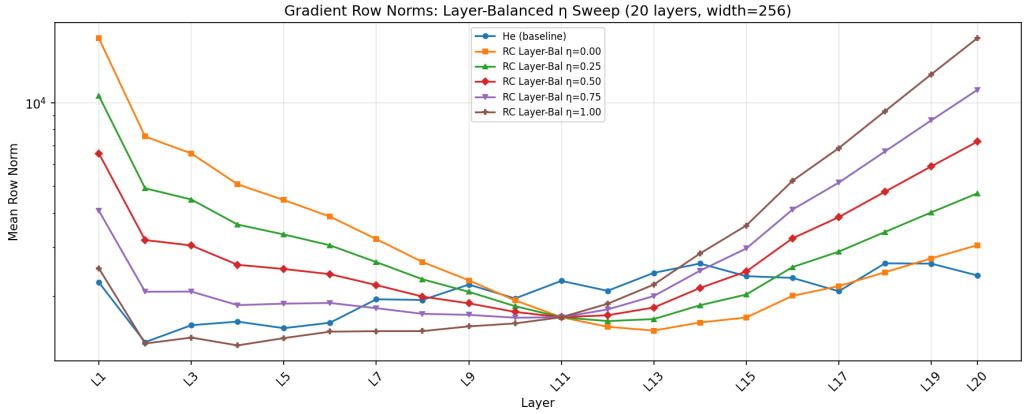

Figure 4.2: Gradient row norms (log scale) for layer-balanced initialization with $\eta \in \{0, 0.25, 0.5, 0.75, 1.0\}$ and He baseline. $\eta = 0.5$ achieves the flattest profile among row-centered variants. Source: `notebooks/06_gradient_diagnostics.ipynb`, Cell 23.

4.2.6 Why $\eta = 1$ overshoots

At $\eta = 1$, the per-layer scaling $s_l = r^{l-(L+1)/2}$ fully compensates forward decay. But it also makes the backward gain **non-uniform**:

- Early layers: $s_l > 1$ (boosted weights) $\Rightarrow$ backward gain $= 1.21 \times s_l \gg 1$ (up to $\sim$7 in the empirical plot).
- Late layers: $s_l < 1$ (reduced weights) $\Rightarrow$ backward gain $= 1.21 \times s_l < 1$.

The layer-dependent s_l that fixes the forward chain **destabilizes the backward chain**. At $\eta = 0.5$, the partial compensation balances forward improvement against backward destabilization.

4.2.7 Comparison to partial centering

Approach	Gradient spread	Geometry	Mechanism
Partial centering ($\alpha = 0.25$)	6.6×	Poor	Reduce centering
Layer-balanced ($\eta = 0.5$)	4.32×	Best (full RC)	Redistribute variance

The layer-balanced approach is **strictly better on the Pareto frontier**: it achieves lower gradient spread (4.32× vs. 6.6×) while maintaining full row-centered geometry.

4.2.8 Fundamental limitation

The backward gain of 1.21 (from the forward-balanced base) limits how much the layer-balanced approach can improve. The η parameter redistributes gradient non-uniformity between forward and backward chains but cannot eliminate it. The optimal $\eta \approx 0.5$ represents the point where forward compensation and backward destabilization are balanced.

Chapter 5

Geometry Revisited: The PCA Metric Was Wrong

Experimental setup. Fashion-MNIST (2000 samples), depths $L \in \{5, 10, 15, 20\}$, width 784 (square projections). Five initializers compared using the three geometry metrics from Section 1.3.

5.1 k-NN Results Overturn the Narrative

Initializer	5 L	10 L	15 L	20 L
`he`	0.736	0.710	0.673	**0.639**
`orthogonal_he`	**0.755**	**0.724**	**0.696**	**0.662**
`orthogonal_tuned`	0.755	0.724	0.696	0.662
`row_centered_he_var_adj`	0.717	0.482	0.186	0.110
`kernel_preserving`	0.737	0.628	0.540	0.100

Table 5.1: k-NN accuracy ($k = 5$) vs. depth. Chance level = 0.10 (10 classes).

The key finding: Row-centered var-adj drops from 0.717 (5 L) to **0.110** (20 L) — indistinguishable from **chance level**. Meanwhile He degrades gracefully from 0.736 to 0.639, losing only ∼13% of its class separability over 20 layers.

This directly contradicts the PCA scatter plot evidence from earlier work, which suggested row centering "preserves geometry" better than He.

Initializer	5 L	10 L	15 L	20 L
`he`	0.843	0.767	0.690	0.654
`orthogonal_he`	**0.875**	0.728	0.670	0.600
`row_centered_he_var_adj`	0.743	0.476	0.363	0.349
`kernel_preserving`	0.722	0.598	0.524	NaN

Table 5.2: Pairwise distance correlation (Spearman) vs. depth.

5.2 Why the PCA Metric Was Misleading

The effective dimensionality reveals the mechanism:

- **Row-centered**: $d_{\text{eff}} = 388$ at 20 L (data spread across ∼388 dimensions).

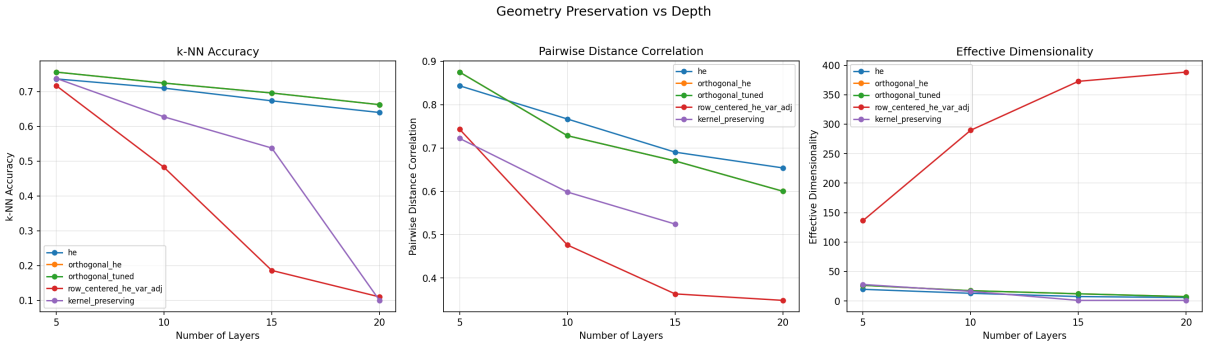

Figure 5.1: Three geometry metrics vs. depth for five initializers. **Left:** k-NN accuracy — row-centered drops to chance (0.10) by 20 layers while He stays at 0.64. **Center:** Pairwise distance correlation. **Right:** Effective dimensionality — row-centered has high d_{eff} (spread-out noise) despite zero class structure. Source: `notebooks/07_kernel_geometry_analysis.ipynb`, Cell 6.

Initializer	5 L	10 L	15 L	20 L
`he`	19.7	13.0	7.5	5.8
`orthogonal_he`	26.6	17.5	12.2	7.3
`row_centered_he_var_adj`	**136.2**	**289.7**	**372.7**	**388.3**
`kernel_preserving`	28.0	16.3	1.0	1.0

Table 5.3: Effective dimensionality (eigenvalue entropy) vs. depth.

- **He**: $d_{\text{eff}} = 5.8$ at 20 L (data compressed into ∼6 dimensions).

The PCA scatter plots showed row-centered data "spreading out" in the first two principal components — which *looks like* preserved structure. But **spread ≠ structure**:

1. The forward signal decays by 0.826^L per layer (equation (3.8)).
2. By 20 layers: $0.826^{20} \approx 0.022$, so activations are ∼46× smaller than the input.
3. At this scale, the signal is dominated by random fluctuations from weight initialization, not input structure.
4. The data becomes **uniformly dispersed noise**: high dimensional (high d_{eff}) but with *no class information* (k-NN = chance).

He, by contrast, compresses data into fewer dimensions (low d_{eff}), but the **class structure is preserved within those dimensions**. The angular contraction from the ReLU kernel ($D(\alpha) > 0$) compresses representations but maintains relative ordering of inter-class vs. intra-class distances.

Remark 5.1 (Effective dimensionality is misleading in isolation)**.** High d_{eff} combined with low k-NN accuracy indicates uniformly dispersed noise, not good geometry. Low d_{eff} combined with high k-NN accuracy indicates efficient compression with preserved class structure. The k-NN accuracy is the correct primary metric.

5.3 Orthogonal Initialization Analysis

5.3.1 Same expected kernel as He

Proposition 5.2. *For a square orthogonal matrix $W = \sqrt{2}Q$ (where Q is from QR decomposition), the expected kernel per neuron is the same as for i.i.d. Gaussian weights with matching*

variance.

Proof sketch. Each row $\boldsymbol{w}_k$ of the orthogonal matrix, viewed in isolation, is uniformly distributed on the sphere of radius $\sqrt{2}$. This is the same distribution as a normalized Gaussian vector $\sqrt{2}\,\boldsymbol{g}/\|\boldsymbol{g}\|$ where $\boldsymbol{g} \sim \mathcal{N}(\mathbf{0}, I)$. In the large-$d$ limit, $\|\boldsymbol{g}\| \approx \sqrt{d}$ concentrates, so the marginal distribution of $\boldsymbol{w}_k$ approaches $\mathcal{N}(\mathbf{0}, (2/d)\, I)$ — the same as He.

The kernel proof (Theorem 1.1) computes $\mathbb{E}[\mathrm{ReLU}(\boldsymbol{w}_k^\top \boldsymbol{x}_i) \cdot \mathrm{ReLU}(\boldsymbol{w}_k^\top \boldsymbol{x}_j)]$, which depends only on the marginal distribution of individual rows. Since orthogonal rows have the same marginal as Gaussian rows, the expected kernel per neuron is identical. □

What orthogonal changes: The rows are no longer independent (they are mutually orthogonal), so the *variance* of the kernel estimate decreases (better concentration). The *expectation* is still $K(\alpha)$ with the same distortion $D(\alpha) > 0$.

Prediction confirmed: Orthogonal shows the same systematic collapse rate as He, with ∼3.6% improvement from reduced noise (0.662 vs. 0.639 at 20 L).

5.3.2 Orthogonal_he equals orthogonal_tuned

With square matrices (784×784) and the same seed, QR produces the same orthogonal matrix Q. The only difference is the gain: $\sqrt{2}\,Q$ vs. $\sqrt{1.65}\,Q$. Since all three geometry metrics (k-NN, Spearman distance correlation, effective dimensionality) are **scale-invariant**, the two produce identical results.

5.4 Kernel-Preserving Failure at Depth

Depth	k-NN	Dist. corr.	d_{eff}	Status
5 L	0.737	0.722	28.0	Comparable to He
10 L	0.628	0.598	16.3	Degrading
15 L	0.540	0.524	1.0	Numerical collapse
20 L	0.100	NaN	1.0	Overflow

The kernel-preserving optimizer (Section 2.5.2) optimizes each layer independently via 200 Adam steps. It does not account for how previous layers have already transformed the data.

- At 5 L: comparable to He (0.737 vs. 0.736).
- At 15 L: $d_{\text{eff}} = 1.0$ — data has collapsed to a single direction.
- At 20 L: k-NN = 0.100 (chance), distance correlation = NaN (float64 overflow from accumulated weight inflation of $\sim 15^{20}$).

Per-layer optimization is **non-composable**: each layer is locally optimal, but the composition of 15+ independently-optimized layers diverges.

5.5 Revised Narrative

The previous argument was:

> *"Row centering preserves geometry but creates a gradient trap."*

The corrected argument is:

Row centering prevents angular collapse but destroys class structure (via forward signal decay) AND creates a gradient trap. These are the same phenomenon — the forward decay kills both.

Row centering:

1. Does **NOT** preserve geometry (as measured by class separability / k-NN accuracy).
2. **DOES** prevent angular collapse (data stays spread out — high d_{eff}).
3. But "staying spread out" $\neq$ "preserving structure." The data gets randomized into high-dimensional noise.

Chapter 6

Can We Fix the Kernel Instead of the Weights?

The previous chapters showed that constraining the weights (row centering) kills the forward signal. This chapter explores two alternative approaches: modifying the ReLU nonlinearity via a negative bias, and analyzing the solutions found by direct kernel optimization.

6.1 Biased ReLU Kernel $K_\beta(\alpha)$

6.1.1 Motivation

Instead of constraining W, add a negative bias $b < 0$ to raise the ReLU threshold: $\text{ReLU}(z + b)$ fires only when $z > -b > 0$. This kills weakly-positive activations (which contribute to DC drift) while keeping the weight matrix **unconstrained** — no zero-sum rows, so no gradient trap from the weight structure.

6.1.2 Definition

Let $(\boldsymbol{x}_i, \boldsymbol{x}_j)$ be unit vectors with angle α. For a weight vector $\boldsymbol{w} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I)$ and normalized bias $\beta = b/\sigma$, the biased kernel is:

$$K_\beta(\alpha) = \mathbb{E}\big[\text{ReLU}(u + \beta) \cdot \text{ReLU}(v + \beta)\big], \tag{6.1}$$

where $(u, v) \sim \mathcal{N}\big(\mathbf{0}, \left[\begin{smallmatrix} 1 & \cos\alpha \\ \cos\alpha & 1 \end{smallmatrix}\right]\big)$.

At $\beta = 0$, this reduces to the standard arc-cosine kernel (Theorem 1.1 with $\sigma = 1$).

6.1.3 Optimization objective

Define the normalized distortion:

$$D_\beta(\alpha) = \frac{K_\beta(\alpha)}{K_\beta(0)} - \cos\alpha. \tag{6.2}$$

We seek β^* minimizing the integrated squared distortion:

$$\beta^* = \arg\min_{\beta \le 0} \int_0^\pi D_\beta(\alpha)^2 \, d\alpha. \tag{6.3}$$

6.1.4 Monte Carlo evaluation

Since there is no known closed-form for $K_\beta(\alpha)$ when $\beta \neq 0$, we evaluate it via Monte Carlo with 10^6 samples. At $\beta = 0$, the MC estimate matches the analytical $K(\alpha)$ to within 1.3% relative error, confirming sufficient accuracy.

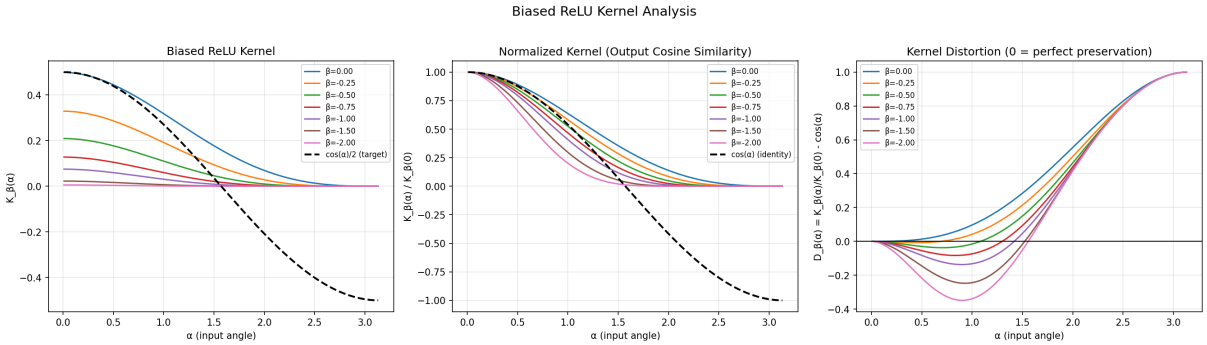

Figure 6.1: Biased ReLU kernel analysis. **Left:** Raw kernel $K_\beta(\alpha)$ for various β. **Center:** Normalized kernel (output cosine similarity). **Right:** Kernel distortion $D_\beta(\alpha)$ — negative bias reduces distortion at small angles but overcorrects at large angles. Source: `notebooks/07_kernel_geometry_analysis.ipynb`, Cell 11.

6.1.5 Results

β	Integrated distortion	Survival rate $\Phi(\beta)$	σ^2_{adj} factor
0.00	0.912	50.0%	1.00
-0.50	0.843	30.9%	1.62
-0.93	**0.795**	**17.6%**	**5.72**
-1.50	0.847	6.7%	14.9
-2.00	0.905	2.3%	43.5

Optimal bias: $\beta^* \approx -0.93$. **Distortion reduction:** $0.795/0.912 = 0.87$, i.e., only a **13% improvement**.

6.1.6 Why it doesn't work

The distortion profiles reveal the mechanism:

- For **small angles** ($\alpha < 1$): negative bias reduces distortion (nearby vectors become less artificially similar).
- For **large angles** ($\alpha > 2$): negative bias **overcorrects** — vectors that were nearly orthogonal appear negatively correlated in the output.
- No value of β reduces distortion uniformly across all angles.

6.1.7 Variance compensation

At $\beta^* = -0.93$, only 17.6% of neurons fire, so the output signal is much weaker. To maintain the same forward signal strength as standard He:

$$\sigma^2_{\text{adj}} = \frac{2/d}{2\,\Phi(\beta^*)} \cdot C(\beta^*)^{-1} \approx 5.72 \cdot \frac{2}{d},$$

where $C(\beta^*)$ is a correction factor from the second moment of the truncated Gaussian.

This 5.72× variance increase would push the backward gain well above 1, potentially creating its own gradient instability.

6.1.8 Assessment

Negative bias is a **dead end**:

1. Marginal improvement (13% distortion reduction) — insufficient to prevent collapse over 20+ layers.
2. Extreme sparsity (82% of neurons silent) requiring massive variance compensation.
3. The distortion is reshaped, not eliminated.
4. **Fundamental limit:** No pointwise nonlinearity (applied independently per neuron) can fully preserve the input kernel. The distortion $D(\alpha)$ is intrinsic to any nonlinear map from $\mathbb{R}^2 \to \mathbb{R}$, and a threshold shift can only change D's shape, not make it zero everywhere.

6.2 Analyzing Kernel-Preserving Optimizer Solutions

6.2.1 Motivation

The `kernel_preserving` initializer (Section 2.5.2) directly minimizes the kernel distortion. At 5 layers it achieves k-NN $= 0.737$ (comparable to He). By analyzing the structure of the optimized weight matrices, we ask:

- Does the optimizer discover row centering?
- Does it discover negative bias?
- Or something entirely different?

We initialize 10 independent layers with `kernel_preserving` and compare the resulting weight matrices to standard He.

6.2.2 Results

Property	He	KP	KP/He ratio
Mean |row sum|	1.2	12.2	10×
Mean row norm $\|\boldsymbol{w}_i\|$	1.3	15.0	11.5×
Weight distribution	Gaussian	Uniform (flat)	—
Mean pre-activation (DC)	0.03	0.39	13×
SVD spectrum	Flat ($\sim$2.5)	Decaying (41 $\to$ 20)	—

Table 6.1: Structural comparison of He vs. kernel-preserving weight matrices.

Flag. The inner-product and norm terms in the KP objective (2.12) are normalized by d_{out}. The norm constraint targets $\|\operatorname{ReLU}(W\boldsymbol{x})\|^2/d_{\text{out}} \approx 1$, which means $\|\operatorname{ReLU}(W\boldsymbol{x})\| \approx \sqrt{d_{\text{out}}}$. This targeting of output norms $\sqrt{d}$ rather than 1 contributes to the observed 11.5× weight inflation: part of the inflation is an artifact of this normalization choice.

6.2.3 Key findings

1. **NOT row centering.** Row sums are 10× *larger* than He. The optimizer moves **away** from zero row sums. If centering were optimal for kernel preservation, the optimizer would have converged toward $\sum_j W_{ij} \approx 0$.

2. **NOT negative bias.** The DC component (mean pre-activation) is 13× larger than He. The optimizer *amplifies* the mean, opposite to the biased ReLU strategy of Section 6.1.

3. **Uniform, not Gaussian.** The weight elements follow a nearly uniform distribution (flat histogram, heavy-tailed Q-Q plot vs. normal), a radical departure from any standard initialization.

4. **Anisotropic SVD.** The singular value spectrum decays from ∼41 (top) to ∼20 (bottom), with condition number ∼ 2× (vs. ∼ 1.1× for He). The optimizer selectively amplifies certain input directions: components along top singular vectors get large pre-activations and survive ReLU with high probability, while components along bottom singular vectors are more likely killed.

5. **Massively inflated weights.** Row norms 11.5× larger than He. This inflation compounds catastrophically at depth: 15^L grows to astronomical values, causing the float64 overflow at 20 layers (Section 5.4).

6.2.4 Why the structure is non-composable

The KP optimizer's anisotropic amplification is tuned to the *input distribution* (random unit vectors). After one KP layer transforms the data, the output distribution is no longer unit vectors — it has been selectively amplified and truncated by ReLU. The next layer's KP optimization, still targeting random unit vectors, operates under violated assumptions.

Over L layers, the accumulated mismatch between the assumed input distribution (unit vectors) and the actual input distribution (increasingly distorted post-ReLU data) causes the optimization to diverge.

6.2.5 Implications

The optimizer confirms that kernel preservation requires a radically different weight structure from any standard initialization. The structure it finds (massive inflation + anisotropy + uniform weights) is inherently per-layer and data-dependent — there is no simple closed-form generalization.

This reinforces the conclusion from Section 6.1: the ReLU kernel distortion is a fundamental property of the nonlinearity that cannot be eliminated by a single-layer weight structure, regardless of its complexity.

Chapter 7

Two Collapse Modes: The Empirical Angle Map

This chapter provides the most intuitive visualization of why row centering fails at geometry despite appearing to succeed in PCA plots.

7.1 Single-Layer Angle Map: All Initializers Are Identical

7.1.1 Method

We generate pairs of random unit vectors at controlled angles $\alpha \in \{5°, 10°, \ldots, 175°\}$ in $\mathbb{R}^{784}$, pass each pair through one initialized layer followed by ReLU, and measure the output angle α' between the resulting vectors. We average over 200 pairs per angle and 5 random seeds.

7.1.2 Results

Initializer	Mean ratio α'/α	Min ratio
`he`	0.7751	0.5081
`orthogonal_he`	0.7754	0.5081
`row_centered_he_var_adj`	0.7751	0.5081
`kernel_preserving`	0.7772	0.5081

All four initializers produce **identical** single-layer angle maps (to within measurement noise). The He empirical result matches the theoretical prediction from the arc-cosine kernel (equation (1.3)) to within 0.14°.

7.1.3 Why row centering is invisible at one layer

The experiment uses random **unit vectors** as inputs. Unit vectors in high dimensions have near-zero mean: $\bar{x} = (1/d)\sum_j x_j \approx 0$. Therefore the DC component is negligible, and the row centering constraint $\sum_j W_{ij} = 0$ has almost no effect on the pre-activations $z_i = \sum_j W_{ij} x_j \approx \sum_j W_{ij}(x_j - \bar{x})$.

Proposition 7.1. *For unit vector inputs with zero DC component, the single-layer angle contraction is determined entirely by the ReLU nonlinearity (through the arc-cosine kernel $K(\alpha)$), independent of the weight structure.*

The differences between initializers emerge only from layer 2 onward, when the inputs are *post-ReLU activations* with a significant positive DC component ($\mathbb{E}[a] = \sigma/\sqrt{2\pi} > 0$).

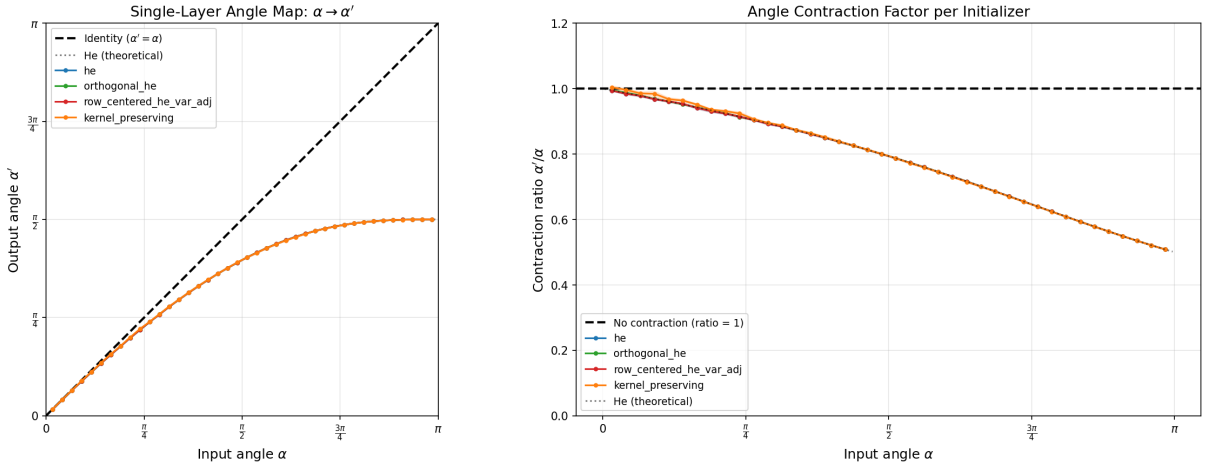

Figure 7.1: **Left:** Single-layer angle map $\alpha \to \alpha'$ for four initializers (empirical, 200 pairs $\times$ 5 seeds) vs. the theoretical arc-cosine kernel prediction. All initializers lie on top of the theory curve. **Right:** Contraction ratio $\alpha'/\alpha < 1$ for all angles. Source: `notebooks/07_kernel_geometry_analysis.ipynb`, Cell 25.

7.2 Multi-Layer Angle Evolution: Two Different Fixed Points

7.2.1 Method

Track the angle between a pair of vectors as they propagate through $L = 20$ layers. Start from four initial angles: $\alpha_0 \in \{30°, 60°, 90°, 120°\}$. Average over 200 pairs and 5 seeds.

7.2.2 Results

Initializer	$30° \to 20L$	$60° \to 20L$	$90° \to 20L$	$120° \to 20L$
`he`	13.8°	17.4°	18.6°	19.0°
`orthogonal_he`	12.5°	15.6°	16.8°	17.3°
`rc_var_adj`	**70.9°**	**71.3°**	**71.3°**	**71.7°**

Table 7.1: Output angles after 20 layers. He converges toward 0°; row-centered converges to $\sim$71°.

He / orthogonal: All starting angles contract toward **0°** — classical geometric collapse where all representations align (*identity collapse*).

Row-centered: All starting angles converge to $\sim$ **71°** — every pair of points becomes equidistant (*equidistance collapse*).

7.3 Why 71° Is the Row-Centered Fixed Point

7.3.1 The mechanism

1. Row centering strips the DC component from post-ReLU activations at each layer (Proposition 3.3).

2. The remaining AC component (directional information) decays by 0.826$\times$ per layer (equation (3.7)).

3. After enough layers, only random fluctuations from the weight matrices remain.

4. The activations at each layer are non-negative (post-ReLU). Random non-negative vectors in high dimension have pairwise angles concentrated around a characteristic value determined by the geometry of the positive orthant.

7.3.2 The positive orthant angle

For random non-negative vectors $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^d_{\geq 0}$ with i.i.d. entries drawn from a half-Gaussian, the expected cosine similarity concentrates (for large d) around:

$$\mathbb{E}\left[\frac{\langle \boldsymbol{a}, \boldsymbol{b} \rangle}{\|\boldsymbol{a}\| \cdot \|\boldsymbol{b}\|}\right] \approx \frac{(\mathbb{E}[a_j])^2}{\mathbb{E}[a_j^2]} = \frac{1/(2\pi)}{1/2} = \frac{1}{\pi} \approx 0.318, \tag{7.1}$$

where we used the half-Gaussian moments (Lemma 3.1) with $\sigma^2 = 1$: $\mathbb{E}[a_j] = 1/\sqrt{2\pi}$, so $(\mathbb{E}[a_j])^2 = 1/(2\pi)$, and $\mathbb{E}[a_j^2] = 1/2$. This gives the expected angle:

$$\alpha^* = \arccos\left(\frac{1}{\pi}\right) \approx 71.5°. \tag{7.2}$$

Remark 7.2. The predicted fixed point (71.5°) matches the empirically observed value ($\sim$71°) to within measurement noise. This excellent agreement confirms the mechanism: after enough layers of DC stripping and signal decay, the post-ReLU activations become effectively random non-negative vectors, and their pairwise angles concentrate at $\arccos(1/\pi)$ — a universal constant determined solely by the geometry of the positive orthant and the half-Gaussian distribution.

7.4 Connecting to the k-NN Results

The two fixed points provide the most intuitive explanation of the geometry results from Chapter 5:

Property	He (fixed point 0°)	RC (fixed point 71°)
Failure mode	All vectors align	All vectors become equidistant
What survives	Relative ordering	Spread (dimensionality)
k-NN at 20 L	0.639 (degrades slowly)	0.110 (chance)
d_{eff} at 20 L	5.8 (collapsed)	388 (high)

Both are geometric collapse, but they destroy **different** aspects of the input geometry:

- **He** preserves **ordering** (which vectors are closer to each other) but loses **scale** (all angles shrink toward 0°). Since k-NN classification depends only on the ranking of distances, not their absolute values, He maintains usable class structure.
- **Row-centered** preserves **scale** (angles stay around 71°, not shrunk to zero) but loses **ordering** (all pairwise angles become identical, so there is no way to tell which points were originally in the same class).

For classification tasks, **ordering is more important than scale** — explaining why He (k-NN = 0.639) vastly outperforms row-centered (k-NN = 0.110) despite having lower effective dimensionality.

Chapter 8

Conclusions and Open Questions

8.1 Summary of Proven Results

1. **The gradient trap is structural** (Chapter 3). Row centering creates a forward gain of $\sqrt{(\pi-1)/\pi} \approx 0.826$ per layer, with a locked forward/backward ratio of 0.827. This was confirmed empirically (single-sample experiments reproduce the same pattern; variance sweep shows constant ratio across all variance factors). No scalar variance adjustment can make both gains equal to 1 simultaneously.

2. **Row centering destroys class structure at depth** (Chapter 5). Using k-NN accuracy as the geometry metric, `row_centered_he_var_adj` drops to chance level (0.110) by 20 layers. Standard He stays at 0.639. The PCA scatter plots that previously suggested geometry preservation were measuring *spread*, not *structure*.

3. **Forward decay kills both geometry and gradients** (Chapters 3–5). These are the same phenomenon: the 0.826^L signal decay randomizes activations at depth (destroying class information) and creates non-uniform gradients (preventing learning).

4. **Partial centering and layer-balanced are partial fixes** (Chapter 4). The alpha sweep reveals a Pareto frontier with no escape. The layer-balanced approach ($\eta = 0.5$) reduces gradient ratio from 11× to 4× while keeping full geometry, but the backward gain of 1.21 limits further improvement.

5. **Negative bias cannot fix the kernel** (Section 6.1). Optimal $\beta^* \approx -0.93$ reduces distortion by only 13%, at the cost of 82% silent neurons and 5.7× variance compensation. No pointwise nonlinearity can fully preserve the input kernel.

6. **The kernel-preserving optimizer confirms centering is wrong** (Section 6.2). The optimizer discovers the *opposite* of row centering (large row sums, inflated weights, anisotropic structure), but this solution is non-composable across layers.

7. **Two different collapse modes** (Chapter 7). He collapses to 0° (identity: all vectors align). Row-centered collapses to 71° (equidistance: all vectors become equidistant). Both destroy class information, but He preserves *ordering* (which k-NN needs) while row-centered destroys it.

8. **Orthogonal_he is the practical winner** (Section 5.3). It achieves k-NN = 0.662 at 20 layers (vs. He's 0.639), a ∼3.6% improvement from reduced variance, with no gradient trap.

8.2 The Real Enemy: $D(\alpha) > 0$

The systematic ReLU kernel distortion

$$D(\alpha) = \frac{1}{\pi}\big(\sin\alpha - \alpha\cos\alpha\big) > 0 \quad \text{for all } \alpha \in (0, \pi)$$

affects **all** initializers. No single-layer weight structure can eliminate it:

- The distortion depends on the normalized kernel $\hat{K}(\alpha) = K(\alpha)/K(0)$, which is invariant to weight scaling.
- Orthogonal matrices have the same marginal distribution as Gaussian, so the same $\hat{K}$.
- Row centering changes the effective kernel but introduces worse problems (forward decay, equidistance collapse).
- Negative bias reshapes $D_\beta(\alpha)$ but cannot make it zero everywhere.
- The kernel-preserving optimizer finds a complex solution that works per-layer but is non-composable.

This distortion is **fundamental to any pointwise nonlinearity**: the operation $\mathbb{E}[\phi(\boldsymbol{w}^\top \boldsymbol{x}_i) \cdot \phi(\boldsymbol{w}^\top \boldsymbol{x}_j)]$ cannot equal $\langle \boldsymbol{x}_i, \boldsymbol{x}_j \rangle$ for all angles unless ϕ is linear.

8.3 Open Questions

1. **Reframing row centering in the thesis.** The professor proposed row centering specifically for geometry preservation. Our results show it is worse than standard He for class separability. How should we present this finding diplomatically while being scientifically rigorous?
2. **Pivoting toward bounding the distortion.** If the kernel distortion cannot be eliminated, perhaps the thesis contribution is proving this impossibility and characterizing the best achievable bounds. What is the optimal single-layer angle map achievable by any weight structure?
3. **Multi-layer or joint optimization.** The kernel-preserving optimizer fails because each layer is optimized independently. Could jointly optimizing all layers (accounting for cumulative kernel drift) produce a composable solution?
4. **Alternative activation functions.** The distortion $D(\alpha) > 0$ is specific to ReLU. Activation functions with $\mathbb{E}[\phi(z)] = 0$ (e.g., centered ReLU, GELU minus its mean, or antisymmetric activations) would eliminate the DC problem that drives the row-centering failure. Do they also reduce the kernel distortion?
5. **Practical implications.** For networks trained with gradient descent (not just random projections), does the initialization-time geometry matter? The gradient trap prevents early learning, but adaptive optimizers (Adam, LARS) may compensate.

Bibliography

[1] Youngmin Cho and Lawrence K Saul. Kernel methods for deep learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 22, 2009.

[2] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010.

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.